



Level 0x0f

Reverse Conference



Topics

- Events
- Code Quest

- 
- Reverse
 - Chris Domas - multiple failures
 - Jordan Moore - one way munitions
 - Laurie Kirk - deobfuscation / compilers
 - Memcpy
 - Timing Glitches
 - DMA
 - Game cheating
 - Game hacking with my sprite editor

Cyber Quest and Code Quest

- Code Quest
 - Wildcat 1 - 2nd place! (540 points)
 - Wildcat 2 - 7th in division, 9th at site (195 points)
 - 97 teams competed this year in Orlando
 - Biggest site for Code Quest in the world
 - [Survey](#)
- Spring Break
 - March 23rd - 27th
- Cyber Quest
 - Saturday, March 28th
 - Registration Jan 5th -
- AI Quest - April 11th
 - Registration deadline March 24th





Cyber Quest

Saturday March 28th

Schedule is TBD / Tentative

U.S. In-Person Lockheed Martin CYBERQUEST® Events (Local Time)

8:00 A.M.	Team Arrivals & Registration
8:30 A.M.	Workstation Set-Up & Light Breakfast
9:30 A.M.	Welcome - Introduction and Kick-Off Competition Begins
10:00 A.M.	Coaches Corner - Program of activities and presentations for the coaches during the competition
1:00 P.M.	Competition Ends - Lunch, Speakers, Other Activities
1:45 P.M.	Awards Ceremony
2:00 P.M.	Event Conclusion and Departure

Wildcat 1

Status: Confirmed

Students: Eshan Vipul - Jay Jackson - Parker Wales - Zachary Wales
- Charles Levitt

Alternate Coach: Debra Jerdon

[Edit](#)[Assign Students](#)[Get Login Info](#)[Withdraw](#)

Wildcat 2

Status: Documents Needed

Students: Wyatt Dangler - Ian Burgas Pizarro - Jasper Bartlett -
Nathan de Wall - Kirin Reiter

Alternate Coach: Debra Jerdon

[Edit](#)[Assign Students](#)[Get Login Info](#)[Withdraw](#)

Lockheed Martin CYBERQUEST® - Technical Information

- For in-person events:
 - Each team is allowed to bring up to three (3) laptop computers, and may also bring one mouse, one monitor, and/or one keyboard for each computer, unless otherwise specified by individual sites.
 - Teams are prohibited from using virtual machines not provided by Lockheed Martin.
 - Teams are prohibited from live streaming the competition on **any** platform (e.g. YouTube, Twitch, Kick, TikTok, etc.)
 - Very large monitors are discouraged due to space limitations and power consumption issues. See site-specific rules for more details.
 - Computers must be capable of connecting to an 802.11x wireless network. If using a school-issued computer, verify that it is able to connect to unrecognized networks prior to attending the event.
- For virtual events:
 - Each student may utilize their own computer.
 - Teams are prohibited from using virtual machines not provided by Lockheed Martin.
 - Teams are prohibited from live streaming the competition on **any** platform (e.g. YouTube, Twitch, Kick, TikTok, etc.)
 - Virtual teams will be provided a virtual meeting communication method during the competition.
- Tools and software required to complete the competition will be provided on the systems within the challenge environment.
- ChatGPT and similar AI platforms are not prohibited from use during the competition. However, students and teams are prohibited from entering Lockheed Martin CYBERQUEST® challenge questions from the competition into any AI platform, including ChatGPT, **and doing so may lead to disqualification.**

Lockheed Martin CYBERQUEST® - Hardware/Software Requirements

Computers used for the competition must meet the following requirements to ensure that students are able to fully participate all challenges.

Chromebooks are not supported due to compatibility issues with the competition's environment and will not work to connect to the competition environment.

- If using Microsoft Windows:
 - 64-bit Windows 10 or higher
 - x64 processor
 - .NET Framework 4.7.2 or higher
 - Access to Microsoft Remote Desktop Connection
 - SSH Client (PuTTY is recommended, but any SSH client should work)
- If using macOS:
 - 64-bit macOS Ventura (13.7), Sonoma (14.7), Sequoia (15.5)
 - Access to Windows App for Mac (previously named Microsoft Remote Desktop)
 - Access to terminal and built-in SSH client
- If using Linux:
 - AMD64 Ubuntu 22.04 LTS (Jammy Jellyfish) or Ubuntu 24.04 LTS (Noble Numbat)
 - Remote Desktop Client (Remmina is recommended)
 - Access to terminal with an installed SSH client
- Must have an up-to-date version (updated within 1 month of the competition) of Google Chrome or Mozilla Firefox
- Must be able to install and run a VPN client to connect to the competition environment



Security Notices

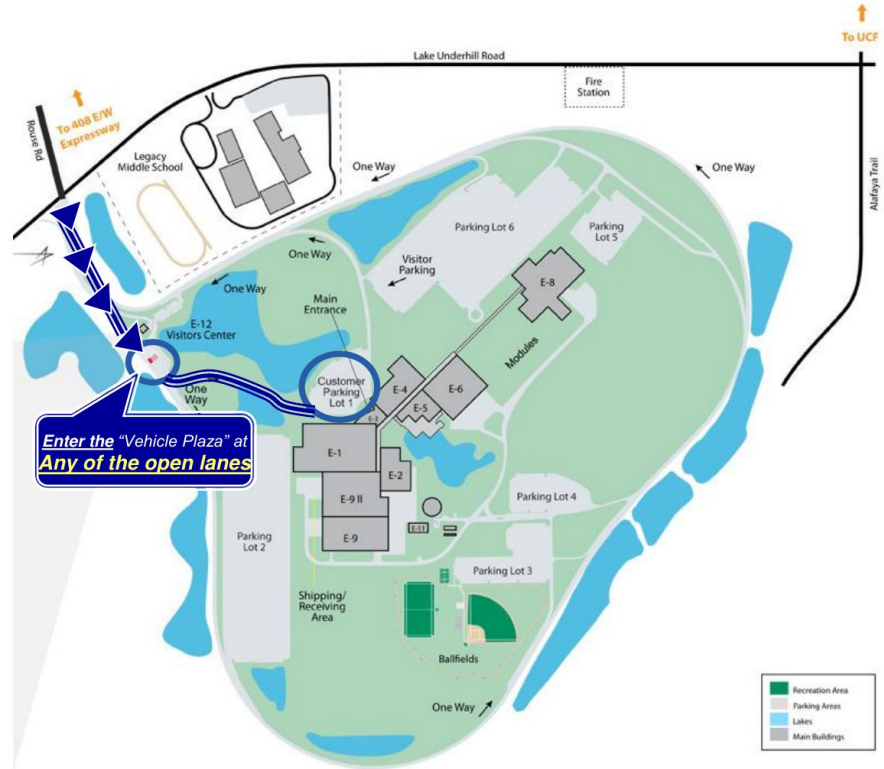
- At Security Gate, say code word “Code Quest.” (Please inform bus drivers, parent drop off/pick up, coaches/students driving to competition).
- Photo IDs are required for all coaches and students.
 - Coaches, non-U.S. students, and Permanent Residents are required to bring valid government-issued photo I.D. and original documentation (green card, passport, visa, resident card).
 - All students should bring a valid form of I.D. (school I.D. is acceptable).
- All phones must be left in your cars during the Code Quest event. Under no circumstances are any phones or recording devices allowed inside Lockheed Martin facilities. This applies to coaches and students.
- In case of emergency, family can call/text Michelle Begonia (321-230-5695) or Abbie Mitchell (407-446-9871) on the day of the competition.




LOCKHEED MARTIN
Training and Logistics Solutions
100 Global Innovation Circle
Orlando, FL 32825-5003
(407) 306-1000



- Drive onto Lockheed Martin Facility at Rouse Rd & Lake Underhill
- Enter the “Vehicle Plaza” at any of the open lanes.
- Stop & Check in with Security Guard who will grant you access to the facility.
- Parking - Follow road on the LEFT for Parking Lot #1.





More Rules

- No spectators will be permitted access to the competition. Only registered students and coaches may attend the event.
- Breakfast, lunch, and drinks will be provided. Dietary and allergic restrictions cannot be accommodated. You may bring your own breakfast, lunch, and water bottle as needed.
- Each team (not person) can bring only one laptop
 - A limited number of loaner laptops are available. If your team needs a laptop, please contact Michelle Begonia (codequest-orlando.gr-ebs@lmco.com).
 - **For security reasons, laptops and other devices manufactured by Lenovo are not permitted at this facility. Please do not bring any Lenovo devices with you.**
- All teams must RETURN problem packet to designated box prior to exiting the facility.
- Coaches and students should not post Code Quest competition questions and/or answers online. Questions will be made available on Code Quest Academy after the competition.



Chris Domas

fail: jmp fail

Resume: Mr movfuscator, sandsifter, many others

Presentation of RE Lessons Learned

- Side Quests for IRL video game
- Measuring the number of clock cycles for single instruction
 - Helped fine tune timing based attacks
- Power profiling functions
 - Using the end of a long integration window to get finer data points
- DMA PCI cards
- You learn new techniques and add tools to your toolbox even when you fail
- “I’ve never been so humbled by someone's list of failures”





Jordan “lockbox” Moore

One Way Munitions

Resume: Zealot Labs, Styx Emulator

Presentation on Suicide Drones



- Buys Shahed / Geran-2 drone components from Ukrainians
 - USA cloned (LUCAS) and added Starlink and jet engine
- ISIS tracks parts: <https://isis-online.org/> (Institute for Sci and Int'l Security)
 - Up to 450 sent into Ukraine per day, 200 on average
- war-sanctions.gur.gov.ua
 - Lots of TI parts, Raspberry Pis, STM32, and Jetson SBC/uControllers
- Led to some fun lighthearted lunch time talk at the table NOT
- “How do you get around import / export restrictions?” “YOLO?”

 **LIVE** Streaming now Shop exclusive items from trusted sellers

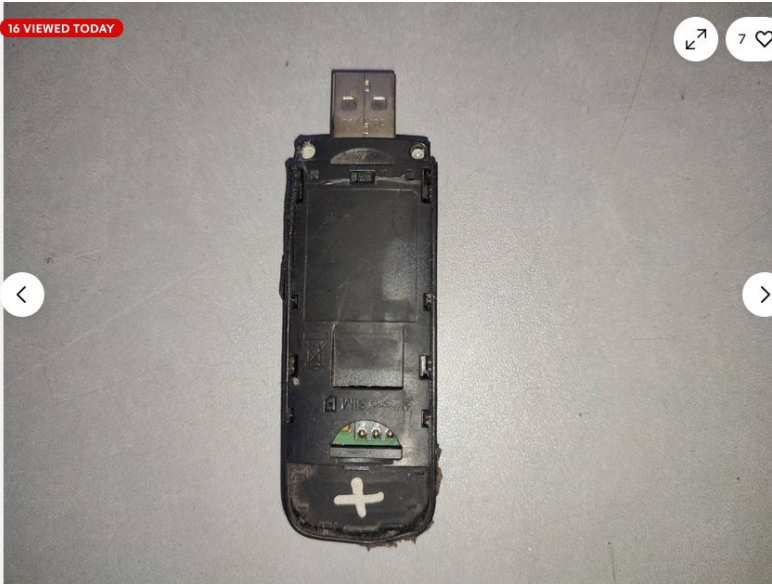
[Join event](#)



16 VIEWED TODAY



7



Original Russian Army Russia Armed Forces Military Drone Shahed GERAN-2 Modem



Stay with UA (945)

100% positive · [Seller's other items](#) · [Contact seller](#)



US \$99.00 or Best Offer

or 4 interest-free payments of \$24.75 available with **Klarna**. [Learn more](#)

Condition: **Used** ⓘ

Buy It Now

[Add to cart](#)

[Make offer](#)

[Add to Watchlist](#)



People are checking this out. 7 have added this to their watchlist.



Breathe easy. Returns accepted.

Have one to sell?

[Sell one like this](#)

[Sell something else](#)

[Share](#)

Shipping: **US \$25.00** Economy Shipping from outside US. [See details](#)

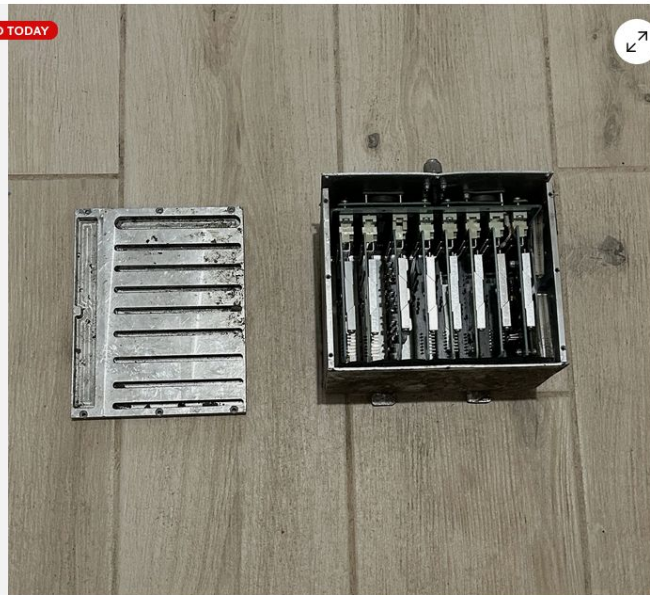
Located in: EU, Ukraine

 **LIVE** Streaming now Shop exclusive items from trusted sellers

[Join event](#)



19 VIEWED TODAY



2 

Shahed 136 / Geran Drone Electronic Control Box – Authentic Part



movsu-66 (109)

100% positive · [Seller's other items](#) · [Contact seller](#)



US \$450.00 or Best Offer

as low as \$40.40/mo with **Klarna**. [Learn more](#)

Condition: **Used** ⓘ

[Buy It Now](#)

[Add to cart](#)

[Make offer](#)

[♥ Add to Watchlist](#)



People are checking this out. 2 have added this to their watchlist.

Shipping: **US \$10.00** USPS Ground Advantage®. [See details](#)

Located in: Zaporozhye, Ukraine

i Due to US customs policies, the buyer of this item will need to pay import fees to the shipping carrier prior to delivery. [Learn more](#)

Have one to sell?

[Sell one like this](#)

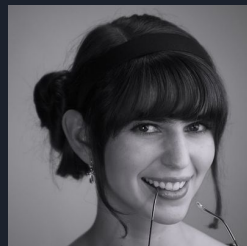
[Sell something else](#)

[Share](#)

Laurie Kirk, James McGowan, Bas Zweers

Obfuscation / Deobfuscation

- LLVM optimization passes
 - Programs can add obfuscation during compilation
- Binary arithmetic to mutilate disassembly / decompilation
 - $A + B = A \wedge B + 2(A \& B)$
- Create tools to identify the mutations / patch to something sane
- Laurie Kirk aka Laurie Wired
 - A lot of background in compilation process
 - Linking process
 - Obfuscators and how they work with LLVM
- How video game anti-cheat works
 - The gaming anti-cheat people didn't like...?



❖ This extends beyond 1-bit-space.

$$x + y \cong \begin{array}{|c|} \hline 0 \\ \hline 1 \\ \hline 1 \\ \hline 2 \\ \hline \end{array} \cong (x \mid y) + (x \& y)$$

Z3 does not have this as a heuristic.

Markus Gaasedelen

XBox One Hack

- Hardware glitching
- Glitching protections and vulnerabilities
 - Random delays
 - memcpy operations
 - I/O for timing
- A little bit of CS, CpE, and EE all-in-one hack
- Hacked Xbox One like no one has before
 - Bootrom level, unpatchable
 - Helps game preservation and repairability





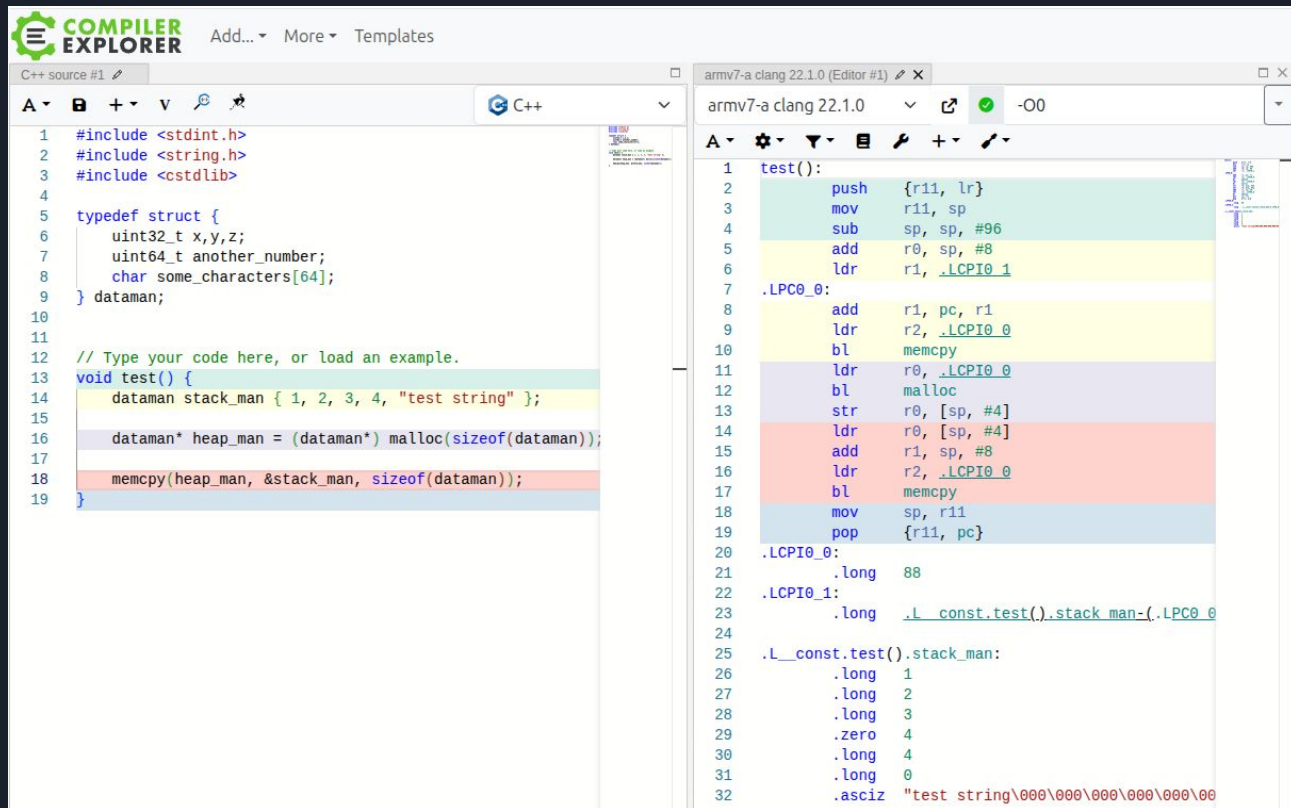
Many more talks

- Video game hacking
- Golang reversing
- Breaking encrypted USB drives <- Hilarious

Check the talks out on Youtube: <https://www.youtube.com/@REverseConf>



Compiler Explorer



The image shows the Compiler Explorer web application. The left pane displays C++ source code for a program named `test()`. The code includes standard headers, defines a `dataman` struct with fields `x`, `y`, `z`, `another_number`, and `some_characters`, and then calls `test()` which allocates memory on the stack and heap, and copies data. The right pane shows the generated ARMv7-A assembly code for the same program, compiled with Clang 22.1.0. The assembly includes instructions for pushing registers, moving the stack pointer, allocating memory, and copying data. The output is color-coded to match the source code.

```
#include <stdint.h>
#include <string.h>
#include <stdlib.h>

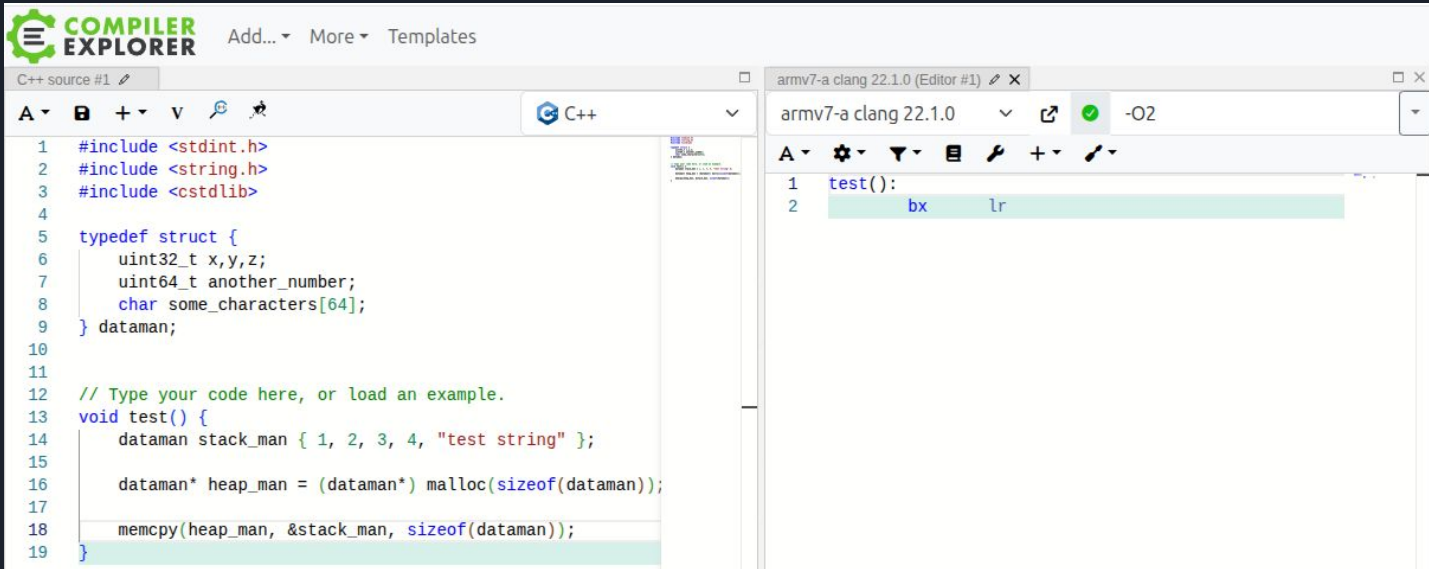
typedef struct {
    uint32_t x,y,z;
    uint64_t another_number;
    char some_characters[64];
} dataman;

// Type your code here, or load an example.
void test() {
    dataman stack_man { 1, 2, 3, 4, "test string" };
    dataman* heap_man = (dataman*) malloc(sizeof(dataman));
    memcpy(heap_man, &stack_man, sizeof(dataman));
}
```

```
test():
    push    {r11, lr}
    mov     r11, sp
    sub     sp, sp, #96
    add     r0, sp, #8
    ldr     r1, .LCPI0_1
.LCPI0_0:
    add     r1, pc, r1
    ldr     r2, .LCPI0_0
    bl      memcpy
    ldr     r0, .LCPI0_0
    bl      malloc
    str     r0, [sp, #4]
    ldr     r0, [sp, #4]
    add     r1, sp, #8
    ldr     r2, .LCPI0_0
    bl      memcpy
    mov     sp, r11
    pop     {r11, pc}
.LCPI0_0:
    .long   88
.LCPI0_1:
    .long   .L_const.test().stack_man-.LCPI0_0
.L_const.test().stack_man:
    .long   1
    .long   2
    .long   3
    .zero   4
    .long   4
    .long   0
    .asciz  "test string\000\000\000\000\000\000"
```

With Optimization -O2

Where is all the assembly? Is this thing broken?



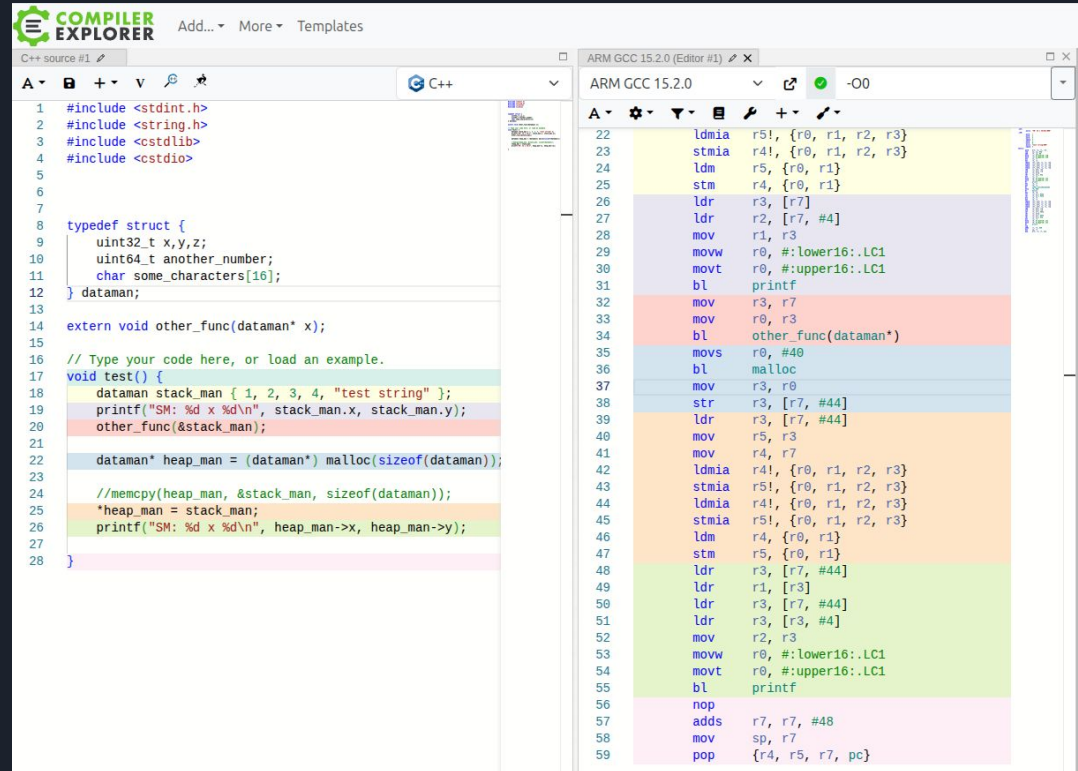
The screenshot displays the Visual Studio Code interface with two panels. The left panel, titled 'C++ source #1', shows a C++ program. The right panel, titled 'armv7-a clang 22.1.0 (Editor #1)', shows the assembly output for the same code, compiled with the -O2 optimization flag.

```
1 #include <stdint.h>
2 #include <string.h>
3 #include <cstdlib>
4
5 typedef struct {
6     uint32_t x,y,z;
7     uint64_t another_number;
8     char some_characters[64];
9 } dataman;
10
11
12 // Type your code here, or load an example.
13 void test() {
14     dataman stack_man { 1, 2, 3, 4, "test string" };
15
16     dataman* heap_man = (dataman*) malloc(sizeof(dataman));
17
18     memcpy(heap_man, &stack_man, sizeof(dataman));
19 }
```

```
1 test():
2     bx     lr
```

Another HW specific way to do memcpy...

- LDMIA
 - Load Multiple Regs
- STMIA
 - Store Multiple Regs
- To copy memory between 2 addresses
 - Load data into a bunch of registers from src addr
 - Store data from registers into dest addr
- More regs = more vulnerable to glitching!



The screenshot displays a code editor with two panes. The left pane shows C++ source code for a program that demonstrates memory copying using LDMIA and STMIA instructions. The right pane shows the corresponding ARM assembly code generated by the compiler.

C++ Source Code (Left Pane):

```
1 #include <stdint.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <stdio.h>
5
6
7
8 typedef struct {
9     uint32_t x,y,z;
10     uint64_t another_number;
11     char some_characters[16];
12 } dataman;
13
14 extern void other_func(dataman* x);
15
16 // Type your code here, or load an example.
17 void test() {
18     dataman stack_man { 1, 2, 3, 4, "test string" };
19     printf("SM: %d x %d\n", stack_man.x, stack_man.y);
20     other_func(&stack_man);
21
22     dataman* heap_man = (dataman*) malloc(sizeof(dataman));
23
24     //memcpy(heap_man, &stack_man, sizeof(dataman));
25     *heap_man = stack_man;
26     printf("SM: %d x %d\n", heap_man->x, heap_man->y);
27 }
28
```

ARM Assembly Code (Right Pane):

```
22 ldmia r5!, {r0, r1, r2, r3}
23 stmia r4!, {r0, r1, r2, r3}
24 ldm r5, {r0, r1}
25 stm r4, {r0, r1}
26 ldr r3, [r7]
27 ldr r2, [r7, #4]
28 mov r1, r3
29 movw r0, #:lower16:.LC1
30 movt r0, #:upper16:.LC1
31 bl printf
32 mov r3, r7
33 mov r0, r3
34 bl other_func(dataman*)
35 movs r0, #40
36 bl malloc
37 mov r3, r0
38 str r3, [r7, #44]
39 ldr r3, [r7, #44]
40 mov r5, r3
41 mov r4, r7
42 ldmia r4!, {r0, r1, r2, r3}
43 stmia r5!, {r0, r1, r2, r3}
44 ldmia r4!, {r0, r1, r2, r3}
45 stmia r5!, {r0, r1, r2, r3}
46 ldm r4, {r0, r1}
47 stm r5, {r0, r1}
48 ldr r3, [r7, #44]
49 ldr r1, [r3]
50 ldr r3, [r7, #44]
51 ldr r3, [r3, #4]
52 mov r2, r3
53 movw r0, #:lower16:.LC1
54 movt r0, #:upper16:.LC1
55 bl printf
56 nop
57 adds r7, r7, #48
58 mov sp, r7
59 pop {r4, r5, r7, pc}
```

DMA

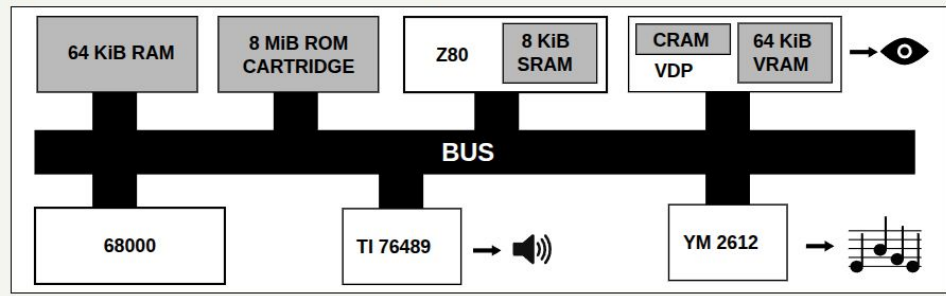
- DMA = Direct Memory Access
 - Allows HW to copy memory without make the CPU do the work!
-
- All graphics for Genesis have to be copied from ROM to VRAM to display on screen
 - DMA transfers help this



The Sega MegaDrive/Genesis. Photo credit: Wikipedia.

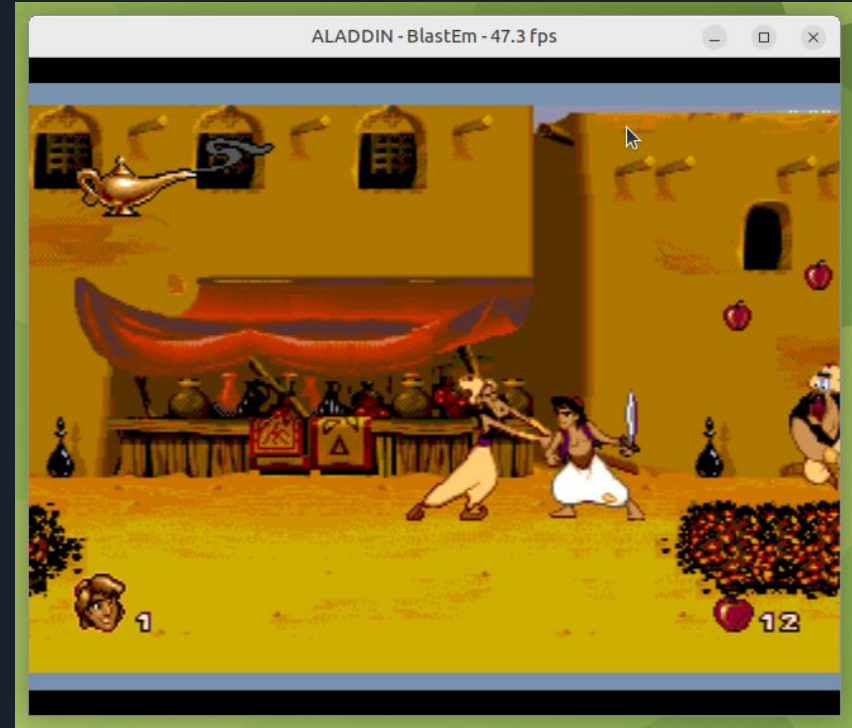
ARCHITECTURE

On the tech side, the Genesis features an impressive Motorola 68000 (7.61MHz), up to 8MiB ROM cartridges, 64KiB RAM, 64 KiB VRAM, a Video Display Processor ASIC (13 MHz), and a Z-80 (3.58 MHz) with 8KiB for audio controlling a TI 76489 and a YM 2612.

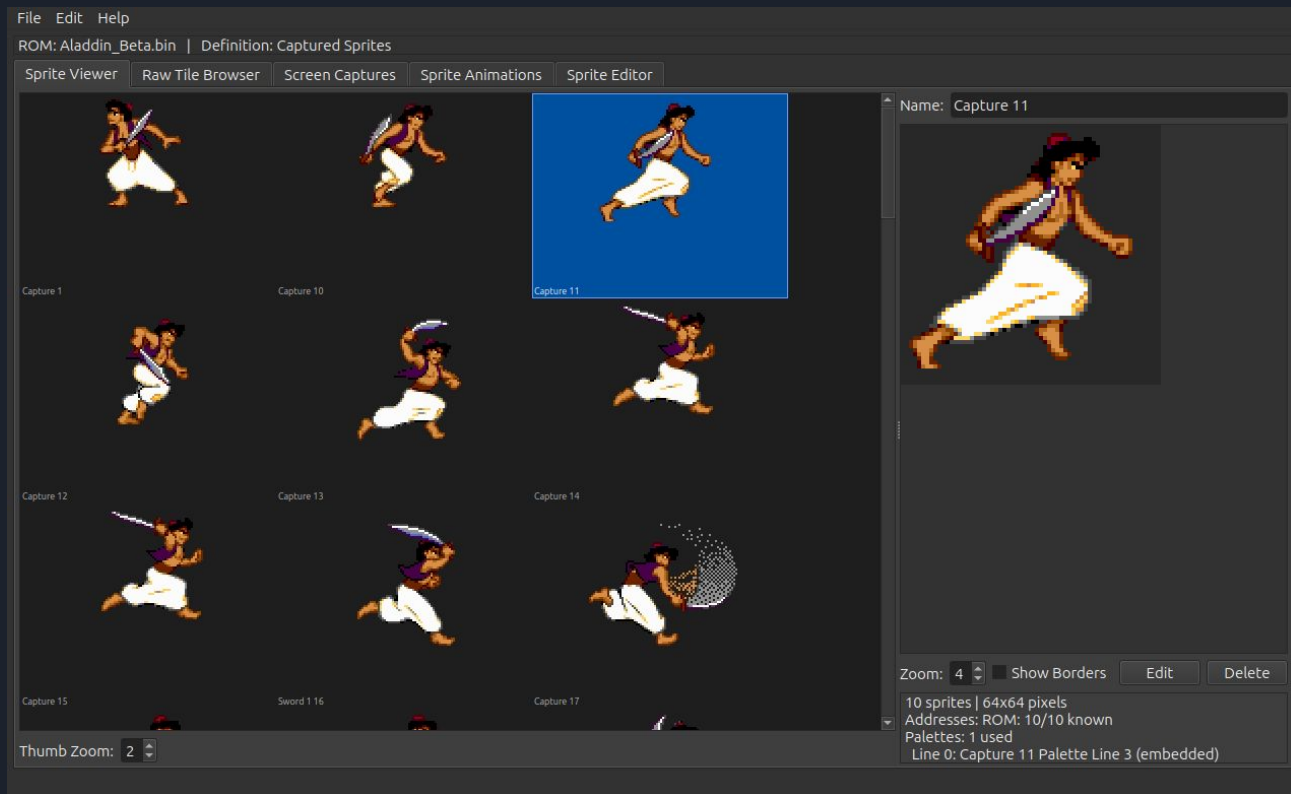


AI Work

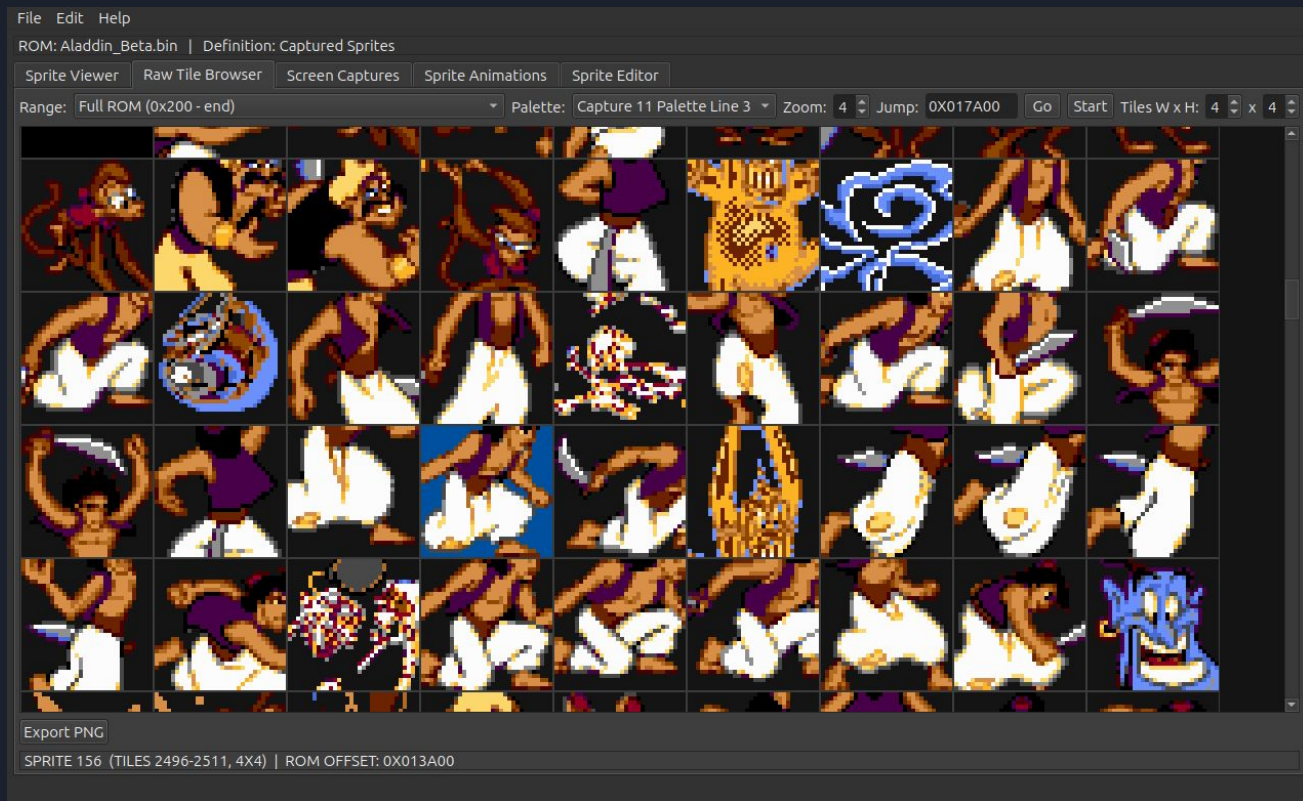
- Want a way to look at and modify artwork from old games
- Emulator Modifications
 - Track DMA transfer operations
 - Sprite recording feature
 - Grabs sprites on screen (the characters that are moving)
 - What memory location did they come from?
- Sprite Viewer / Editor
 - GUI that opens ROM and log to show the individual sprites



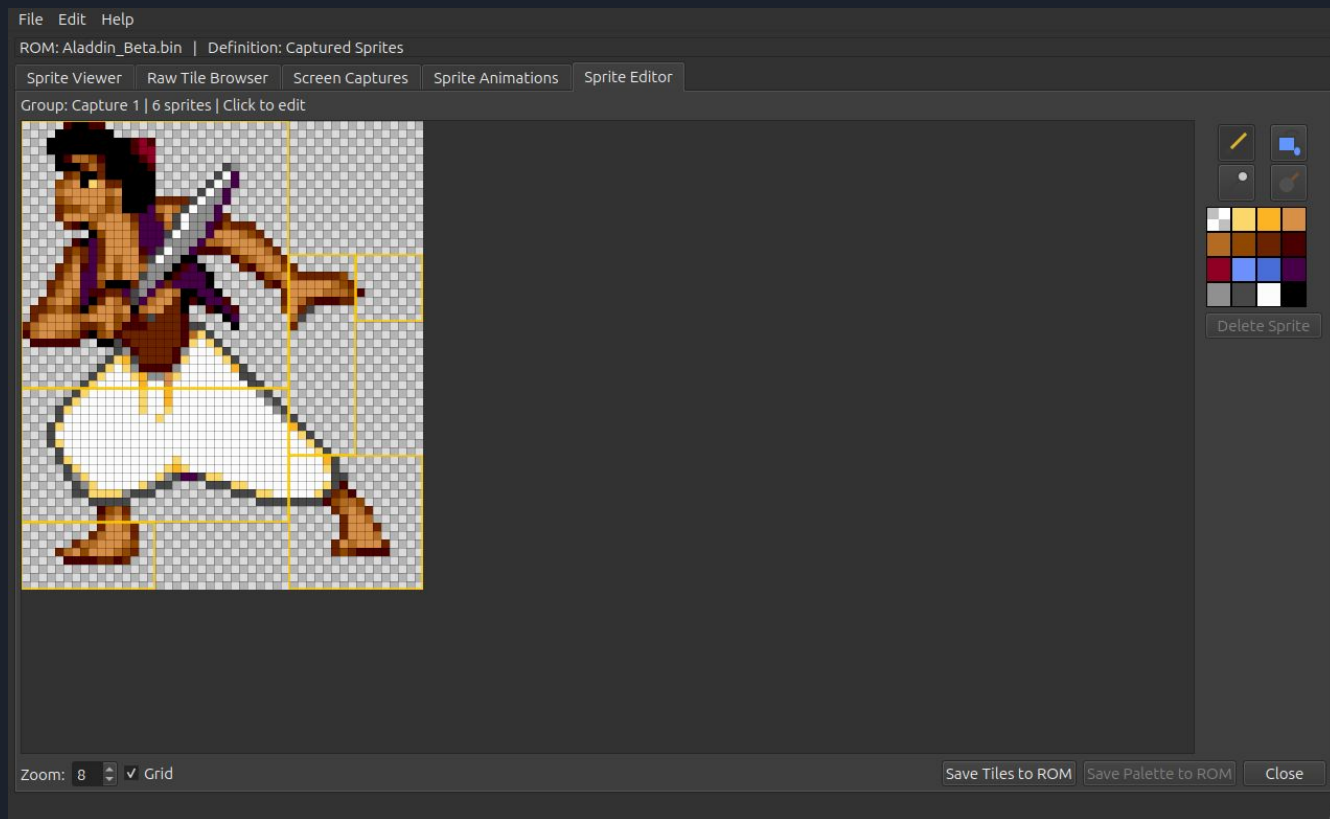
Sprite Sheet Viewer



ROM Tile Browser



Sprite Editor





Agentic AI

- Not talking to the LLM via browser prompt
- Claude Code (Anthropic) and Codex (OpenAI) have applications that run in your computer
 - Can access your files
 - Can access the web, to download and post things
 - Can iteratively work on a project
 - Can work as a team of AI agents together at the same time
- Connecting the AI to other tools
 - MCP servers for Ghidra and Binary Ninja allow them to help reverse engineer and find bugs
- My thoughts so far
 - Can write a lot of code for you
 - Can write a lot of boring documentation and test code!! WIN!
 - They do create some risks, but possibly great rewards
 - Will continue to transform computer science, engineering, and business everywhere...



Links

- https://fabiansanglard.net/another_world_polygons_Genesis/
-